

وواجهات المستخدم Blade Templates: الدرس 4

جدول المحتويات

1. Blade مقدمة في
2. الأساسي Blade بناء
3. Directives التوجيهات
4. Layouts التخطيطات
5. Components المكونات
6. Includes التضمين
7. تمرير البيانات
8. التمارين العملية

Blade مقدمة في

Blade ما هو

Blade (Templating Engine) هو محرك القوالب Blade. بطريقتة ديناميكية وأنيقة HTML يسمح لك بكتابة.

Blade لماذا

بسيط وسهل القراءة

- صيغة نظيفة وواضحة

قوي ومرن

- PHP يدعم جميع مميزات

آمن

- XSS حماية تلقائية من

سريع

- عادي مرة واحدة فقط PHP يتم تحويله لـ

PHP العادي vs Blade

```
<!-- PHP العادي -->
<?php if($user->isAdmin()): ?>
    <h1>Welcome Admin: <?php echo htmlspecialchars($user->name); ?></h1>
<?php endif; ?>

<!-- Blade - أبسط وأوضح -->
@if($user->isAdmin())
    <h1>Welcome Admin: {{ $user->name }}</h1>
@endif
```

Blade أين توجد ملفات؟

```
resources/views/
├── welcome.blade.php
├── home.blade.php
├── layouts/
│   └── app.blade.php
├── components/
│   └── button.blade.php
└── partials/
    ├── header.blade.php
    └── footer.blade.php
```

الأساسي Blade بناء

عرض البيانات 1.

عرض المتغيرات

```
{{ -- عرض متغير -- }}
<h1>{{ $title }}</h1>

{{ -- عرض خاصية من object -- }}
<p>{{ $user->name }}</p>

{{ -- عرض عنصر من array -- }}
<p>{{ $users[0] }}</p>
```

```
{{-- استدعاء function --}}  
<p>{{ strtoupper($name) }}</p>
```

تلقائياً Escape يقوم بـ Blade

```
{{-- إلى نص HTML tags آمن - يحول --}}  
{{ $userInput }} <!-- &lt;script&gt;alert('XSS')&lt;/script&gt; -->  
  
{{-- كما هو HTML خطر - يعرض --}}  
{!! $htmlContent !!} <!-- استخدمه فقط للمحتوى الموثوق -->
```

القيم الافتراضية

```
{{-- إذا كان $name فارغ، يعرض 'Guest' --}}  
<h1>Hello, {{ $name ?? 'Guest' }}</h1>  
  
{{-- أو --}}  
<h1>Hello, {{ $name or 'Guest' }}</h1>
```

2. التعليقات

```
{{-- النهائي HTML لا يظهر في Blade تعليق --}}  
  
<!-- النهائي HTML يظهر في Blade --!> تعليق
```

3. الكود الخام (Verbatim)

```
@verbatim  
    {{-- Blade هذا لن يعالج بواسطة --}}  
    <div>{{ variable }}</div>  
@endverbatim
```

Angular أو Vue.js مفيد عند استخدام

Directives التوجيهات

1. الشروط (Conditionals)

@if, @elseif, @else

```
@if($score >= 90)
    <p class="text-success">ممتاز!</p>
}elseif($score >= 70)
    <p class="text-info">جيد جداً</p>
}elseif($score >= 50)
    <p class="text-warning">جيد</p>
@else
    <p class="text-danger">راسب</p>
@endif
```

@unless

```
{{-- عكس if --}}
@unless($user->isAdmin())
    <p>أنت لست مدير</p>
@endunless

{{-- يعادل --}}
@if(!$user->isAdmin())
    <p>أنت لست مدير</p>
@endif
```

@isset و @empty

```
{{-- التحقق من وجود متغير --}}
@isset($name)
    <p>الاسم: {{ $name }}</p>
@endisset

{{-- التحقق من فراغ متغير --}}
@empty($users)
    <p>لا يوجد مستخدمين</p>
@endempty
```

@auth و @guest

```
{{-- إذا كان المستخدم مسجل دخول --}}
@auth
```

```

    <p>مرحباً {{ auth()->user()->name }}</p>
@endauth

{{-- إذا كان المستخدم زائر --}}
@guest
    <a href="/login">تسجيل الدخول</a>
@endguest

{{-- كلاهما معاً --}}
@auth
    <a href="/dashboard">لوحة التحكم</a>
@else
    <a href="/login">تسجيل الدخول</a>
@endauth

```

2. الحلقات (Loops)

@foreach

```

<ul>
@foreach($users as $user)
    <li>{{ $user->name }}</li>
@endforeach
</ul>

{{-- مع المفتاح والقيمة --}}
@foreach($users as $id => $user)
    <li>{{ $id }}: {{ $user->name }}</li>
@endforeach

```

foreach في \$loop متغير

```

@foreach($products as $product)
    <div class="product"
        @if($loop->first) first-item @endif
        @if($loop->last) last-item @endif
    ">
        <h3>{{ $product->name }}</h3>
        <p>العنصر {{ $loop->iteration }} من {{ $loop->count }}</p>
        <p>المؤشر: {{ $loop->index }}</p>

        @if($loop->odd)
            <span>صف فردي</span>
        @endif
    </div>
@endforeach

```

@endforeach

\$loop: خصائص

الوصف	الخاصية
(المؤشر الحالي (يبدأ من 0	<code>\$loop->index</code>
(التكرار الحالي (يبدأ من 1	<code>\$loop->iteration</code>
العناصر المتبقية	<code>\$loop->remaining</code>
العدد الكلي	<code>\$loop->count</code>
أول عنصر؟	<code>\$loop->first</code>
آخر عنصر؟	<code>\$loop->last</code>
تكرار زوجي؟	<code>\$loop->even</code>
تكرار فردي؟	<code>\$loop->odd</code>
مستوى التداخل	<code>\$loop->depth</code>
(الحلقة الأم (في حالة التداخل	<code>\$loop->parent</code>

@forelse

```
{{-- لكن مع حالة القائمة الفارغة foreach مثل --}}
<ul>
@forelse($users as $user)
    <li>{{ $user->name }}</li>
@empty
    <li>لا يوجد مستخدمين</li>
@endforelse
</ul>
```

@for

```
@for($i = 0; $i < 10; $i++)
    <p>العدد: {{ $i }}</p>
@endfor
```

@while

```
@php $i = 0; @endphp

@while($i < 10)
    <p>{{ $i }}</p>
    @php $i++; @endphp
@endwhile
```

@break , @continue

```
@foreach($users as $user)
    @if($user->type == 'admin')
        @continue
    @endif

    <li>{{ $user->name }}</li>

    @if($loop->iteration >= 10)
        @break
    @endif
@endforeach
```

3. Switch Statement

```
@switch($status)
    @case('pending')
        <span class="badge badge-warning">قيد الانتظار</span>
        @break

    @case('approved')
        <span class="badge badge-success">موافق عليه</span>
        @break

    @case('rejected')
        <span class="badge badge-danger">مرفوض</span>
        @break

    @default
        <span class="badge badge-secondary">غير معروف</span>
@endswitch
```

الخام PHP الـ 4.

```
@php
    $total = 0;
    foreach($items as $item) {
        $total += $item->price;
    }
@endphp

<p>المجموع: {{ $total }}</p>
```

Controller كثيراً. ضع المنطق في @php ملاحظة: تجنب استخدام

Layouts التخطيطات

المفهوم

(Header, Footer, Navigation) هو قالب رئيسي يحتوي على الهيكل العام للصفحة (Layout) التخطيط

1. إنشاء Layout

ملف `resources/views/layouts/app.blade.php`:

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('title', 'الموقع الافتراضي')</title>

    {{-- CSS --}}
    <link rel="stylesheet" href="/css/app.css">
    @stack('styles')
</head>
<body>
    {{-- Header --}}
    <header>
        <nav>
            <a href="/">الرئيسية</a>
            <a href="/about">من نحن</a>
            <a href="/contact">اتصل بنا</a>
        </nav>
```



```

</header>

{{-- Main Content --}}
<main>
    @yield('content')
</main>

{{-- Footer --}}
<footer>
    <p>&copy; 2024 جميع الحقوق محفوظة</p>
</footer>

{{-- Scripts --}}
<script src="/js/app.js"></script>
@stack('scripts')
</body>
</html>

```

2. Layout استخدام

ملف `resources/views/home.blade.php`:

```

@extends('layouts.app')

@section('title', 'الصفحة الرئيسية')

@section('content')
    <h1>مرحباً بك في الصفحة الرئيسية</h1>
    <p>هذا محتوى الصفحة الرئيسية</p>
@endsection

@push('scripts')
    <script>
        console.log('Home page loaded');
    </script>
@endpush

```

@yield vs @section

```

{{-- في Layout --}}
@yield('sidebar')           {{-- محتوى بسيط --}}
@section('content')          {{-- محتوى افتراضي --}}
    <p>المحتوى الافتراضي</p>
@show

```

```

{{-- في الصفحة --}}
@section('sidebar')
    <div>القائمة الجانبية</div>
@endsection

@section('content')
    @parent  {{-- يعرض المحتوى الافتراضي أيضاً --}}
    <p>محتوى إضافي</p>
@endsection

```

@stack و @push

```

{{-- Layout في --}}
<head>
    @stack('styles')  {{-- هنا ستظهر جميع الأنماط --}}
</head>

{{-- في الصفحة 1 --}}
@push('styles')
    <link rel="stylesheet" href="/css/page1.css">
@endpush

{{-- في الصفحة 2 --}}
@push('styles')
    <link rel="stylesheet" href="/css/page2.css">
@endpush

{{-- النتيجة النهائية --}}
<head>
    <link rel="stylesheet" href="/css/page1.css">
    <link rel="stylesheet" href="/css/page2.css">
</head>

```

المكونات Components

أنواع المكونات

1. **Class-based Components** – مكونات PHP Class
2. **Anonymous Components** – مكونات بسيطة

1. Anonymous Components

إنشاء Component

```
php artisan make:component Alert
```

ملف `resources/views/components/alert.blade.php`:

```
@props(['type' => 'info', 'message'])

<div class="alert alert-{{ $type }}">
    {{ $message }}
</div>
```

استخدام Component

```
<x-alert type="success" message="تم الحفظ بنجاح!"></x-alert>

أو

<x-alert type="danger">
    حدث خطأ في العملية
</x-alert>
```

2. Component مع Slot

ملف `resources/views/components/card.blade.php`:

```
@props(['title'])

<div class="card">
    <div class="card-header">
        <h3>{{ $title }}</h3>
    </div>
    <div class="card-body">
        {{ $slot }}
    </div>
</div>
```

الاستخدام:

```
<x-card title="معلومات المستخدم">
```

```
<p>الاسم: أحمد محمد</p>
<p>البريد: ahmed@example.com</p>
</x-card>
```

3. Component مع Slots متعددة

ملف `resources/views/components/modal.blade.php`:

```
@props(['id', 'title'])

<div class="modal" id="{{ $id }}">
    <div class="modal-header">
        <h2>{{ $title }}</h2>
    </div>
    <div class="modal-body">
        {{ $slot }}
    </div>
    <div class="modal-footer">
        {{ $footer }}
    </div>
</div>
```

الاستخدام:

```
<x-modal id="myModal" title="تأكيد الحذف">
    <p>هل أنت متأكد من الحذف؟</p>

    <x-slot name="footer">
        <button class="btn btn-danger">حذف</button>
        <button class="btn btn-secondary">إلغاء</button>
    </x-slot>
</x-modal>
```

4. Class-based Component

```
php artisan make:component Button --class
```

ملف `app/View/Components/Button.php`:

```
namespace App\View\Components;

use Illuminate\View\Component;
```

```

class Button extends Component
{
    public $type;
    public $size;

    public function __construct($type = 'primary', $size = 'md')
    {
        $this->type = $type;
        $this->size = $size;
    }

    public function render()
    {
        return view('components.button');
    }

    public function classes()
    {
        return "btn btn-{$this->type} btn-{$this->size}";
    }
}

```

ملف `resources/views/components/button.blade.php`:

```

<button {{ $attributes->merge(['class' => $classes()]) }}>
    {{ $slot }}
</button>

```

الاستخدام:

```

<x-button type="success" size="lg">حفظ</x-button>
<x-button type="danger" class="mt-2" id="delete-btn">حذف</x-button>

```

Attributes

```

{{-- في Component --}}
<div {{ $attributes }}>
    {{ $slot }}
</div>

{{-- $attributes->merge() --}}
<button {{ $attributes->merge(['class' => 'btn btn-primary']) }}>
    {{ $slot }}
</button>

```

```

{{-- $attributes->class() --}}
<div {{ $attributes->class(['card', 'shadow' => $shadow]) }}>
    {{ $slot }}
</div>

{{-- الاستخدام --}}
<x-button class="my-custom-class" id="submit">إرسال</x-button>

```

Includes التضمين

@include

```

{{-- تضمين ملف --}}
@include('partials.header')

{{-- تضمين مع بيانات --}}
@include('partials.alert', ['type' => 'success', 'message' => 'تم بنجاح!'])

{{-- تضمين شرطي --}}
@includeIf('partials.sidebar')

{{-- تضمين حسب الشرط --}}
@includeWhen($user->isAdmin(), 'partials.admin-menu')

{{-- تضمين إلا في حالة --}}
@includeUnless($user->isGuest(), 'partials.user-menu')

{{-- أول ملف موجود --}}
@includeFirst(['partials.custom', 'partials.default'])

```

مثال عملي

ملف `resources/views/partials/header.blade.php`:

```

<header>
    <nav>
        <a href="/">الرئيسية</a>
        @auth
            <a href="/dashboard">لوحة التحكم</a>
            <a href="/profile">الملف الشخصي</a>
        @endauth
    </nav>
</header>

```

```
@guest
    <a href="/login">تسجيل الدخول</a>
@endguest
</nav>
</header>
```

الاستخدام:

```
@extends('layouts.app')

@section('content')
    @include('partials.header')

    <h1>محتوى الصفحة</h1>
@endsection
```

تمرير البيانات

1. View إلى Controller من

```
// 1 طريقة: compact()
public function index()
{
    $users = User::all();
    $title = 'قائمة المستخدمين';
    return view('users.index', compact('users', 'title'));
}

// 2 طريقة: Array
public function show($id)
{
    return view('users.show', [
        'user' => User::find($id),
        'posts' => Post::where('user_id', $id)->get()
    ]);
}

// 3 طريقة: with()
public function create()
{
    return view('users.create')
        ->with('title', 'إنشاء مستخدم');
```

```
->with('categories', Category::all());  
}
```

2. Views مشاركة البيانات مع جميع

في `app/Providers/AppServiceProvider.php`:

```
use Illuminate\Support\Facades\View;  
  
public function boot()  
{  
    // متاح في جميع Views  
    View::share('appName', 'الموقعي الرائع');  
    View::share('year', date('Y'));  
}
```

View: الاستخدام في أي

```
<footer>  
    <p>&copy; {{ $year }} {{ $appName }}</p>  
</footer>
```

3. View Composers

```
use Illuminate\Support\Facades\View;  
  
public function boot()  
{  
    // واحد View لـ  
    View::composer('dashboard', function ($view) {  
        $view->with('stats', [  
            'users' => User::count(),  
            'posts' => Post::count()  
        ]);  
    });  
  
    // لعدة Views  
    View::composer(['header', 'sidebar'], function ($view) {  
        $view->with('categories', Category::all());  
    });  
  
    // لجميع Views  
    View::composer('*', function ($view) {  
        $view->with('currentUser', auth()->user());  
    });  
}
```



```
});  
}
```

الفرم Forms

CSRF Protection

```
<form method="POST" action="/posts">  
  @csrf {{-- ضروري لحماية الفرمة --}}  
  
  <input type="text" name="title">  
  <button type="submit">إرسال</button>  
</form>
```

Method Spoofing

```
{{-- PUT Request --}}  
<form method="POST" action="/posts/1">  
  @csrf  
  @method('PUT')  
  <button>تحديث</button>  
</form>  
  
{{-- DELETE Request --}}  
<form method="POST" action="/posts/1">  
  @csrf  
  @method('DELETE')  
  <button>حذف</button>  
</form>
```

Validation Errors

```
<form method="POST" action="/register">  
  @csrf  
  
  <div>  
    <label>الاسم</label>  
    <input type="text" name="name" value="{{ old('name') }}">  
    @error('name')  
      <span class="text-danger">{{ $message }}</span>
```

```

        @enderror
    </div>

    <div>
        <label>البريد الإلكتروني</label>
        <input type="email" name="email" value="{{ old('email') }}">
        @error('email')
            <span class="text-danger">{{ $message }}</span>
        @enderror
    </div>

    <button type="submit">تسجيل</button>
</form>

```

Old Input

```

<input type="text" name="name" value="{{ old('name', $user->name) }}">
<textarea name="bio">{{ old('bio', $user->bio) }}</textarea>

```

التمارين العملية

Layout تمرين 1: صفحة مع

Layout:

```

{{-- resources/views/layouts/app.blade.php --}}
<!DOCTYPE html>
<html>
<head>
    <title>@yield('title')</title>
</head>
<body>
    @include('partials.navbar')

    <div class="container">
        @yield('content')
    </div>

    @include('partials.footer')
</body>

```

```
</html>
```

الصفحة:

```
{{-- resources/views/home.blade.php --}}
@extends('layouts.app')

@section('title', 'الصفحة الرئيسية')

@section('content')
    <h1>مرحباً بك</h1>
@endsection
```

Foreach تمرين 2: عرض قائمة مع

```
<h2>المنتجات</h2>

@forelse($products as $product)
    <div class="product">
        <h3>{{ $product->name }}</h3>
        <p>السعر: {{ $product->price }} ريال</p>

        @if($loop->first)
            <span class="badge">جديد</span>
        @endif
    </div>
@empty
    <p>لا توجد منتجات</p>
@endforelse
```

للمر Component: تمرين 3

```
{{-- resources/views/components/button.blade.php --}}
@props(['type' => 'primary'])

<button class="btn btn-{{ $type }}" {{ $attributes }}>
    {{ $slot }}
</button>

{{-- الاستخدام --}}
<x-button type="success">حفظ</x-button>
<x-button type="danger" onclick="confirmDelete()">حذف</x-button>
```

تمرين 4: فورم كامل

```
<form method="POST" action="/posts">
  @csrf

  <div>
    <label>العنوان</label>
    <input type="text" name="title" value="{{ old('title') }}">
    @error('title')
      <span>{{ $message }}</span>
    @enderror
  </div>

  <div>
    <label>المحتوى</label>
    <textarea name="content">{{ old('content') }}</textarea>
    @error('content')
      <span>{{ $message }}</span>
    @enderror
  </div>

  <button type="submit">نشر</button>
</form>
```

ملخص

في هذا الدرس، تعلمت

الشروط والحلقات (Directives): التوجيهات XSS ولماذا نستخدمه عرض البيانات والحماية من Blade ما هو الأجزاء (Includes) وأنواعها التضمين (Components) وإعادة الاستخدام المكونات (Layouts) التخطيطات CSRF تمرير البيانات بطرق مختلفة التعامل مع الفورم و

موارد إضافية

- [Laravel Blade Documentation](#)
- [Blade Components](#)
- [Blade Directives](#)

اختبر نفسك ؟

قبل الانتقال للدرس التالي، تأكد من إجابتك على

1. ما الفرق بين `{ }` و `{!! !}` ؟
2. `@section` ومتى `@yield` متى نستخدم ؟
3. `Include` و `Component` ما الفرق بين ؟
4. `View` إلى `Controller` كيف نمرر بيانات من ؟
5. في الفورم؟ `@csrf` ما فائدة ؟

الدرس التالي

Migrations جاهز للمزيد؟ انتقل إلى **الدرس 5: قواعد البيانات والـ**

في الدرس 5، ستتعلم

- إنشاء وإدارة قواعد البيانات
- `Schema Builder` والـ `Migrations`
- أنواع البيانات المختلفة
- والمزيد!

؟ !تعلم سعيد